



Programação Estruturada em C
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
RELATÓRIO DE AULAS PRÁTICAS

Nome:Vitória Freitas

RA:2523279

Polo de matrícula:Vila Dirce-Carapicuíba

Local da realização da Aula Prática:Alphaville-Santana de Parnaíba

Ano da postagem:2025

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Programação Estruturada em C	Relatório 1
--	--	--------------------

Estrutura de um programa em C

Um programa em C vai ser escrito, utilizando uma ou mais funções. A função principal vai se chamar “main”, cada um dessa função irá retornar algo ou irá retornar 0.

Existem regras para serem seguidas em c, algumas delas seriam: todo programa em C, tem a função main, toda função precisa ter dois parênteses após o nome, as linhas de código no fim recebem um ponto e vírgula, códigos são executados da maneira que foram escritos.

A indentação em C tem sua importância, pois ela gera uma maior legibilidade para o código, mas caso não indente corretamente, o programa não gerará erro por conta disso. Por tanto, indentar em C não é algo obrigatório, mas sim uma boa prática.

Tipos de dados básicos em C

Tipos de dados básicos em C: int(inteiro), char(caracter), double(real de precisão simples), void(vazio).

Comandos básicos em C (printf() e scanf())

Printf() serve para mostrar mensagens na tela, scanf() para ler valores e guardar seu resultado na memória. Para mostrar um valor na tela precisa identificar de que tipo é esse valor, por exemplo se for inteiro precisa do “%d”, float “%f”, char “%c” de um único caractere. Para o programa ler um também precisa identificar o seu tipo, exemplo “&d” para inteiros, “&f” para float, “&c” para char.

Identificadores e variáveis.

Variável tem a função de guardar um valor dentro da memória, todas elas têm identificadores e um tipo.

Identificadores são os nomes que você dá para suas variáveis, eles devem ser diferentes, não pode ter duas variáveis com o mesmo nome. A linguagem C é “case-sensitive”, ou seja, tem diferença uma variável com letra maiúscula e uma com letra minúscula, é obrigatório que eles sejam diferentes dos comandos de C e só pode ser

inicializado com uma letra ou um “_”.

Códigos-Fontes do roteiro, exemplos e atividades

Atividade 1 do roteiro.

```
// Aula 1
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    // variáveis
    char name[50];
    char email[50];
    int age;

    // leitura de dados
    printf("Digite seu nome: ");
    scanf("%s", &name);

    printf("Digite sua idade: ");
    scanf("%d", &age);

    printf("Digite seu e-mail: ");
    scanf("%s", &email);

    // mostrando os dados
    printf("Ola, %s! Voce tem %d anos, seu e-mail: %s\n", name, age, email);

    return 0;
}
```

Outras atividades feitas.

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    // primeira atividade

    printf("Ola, mundo!\n");

    // segunda atividade
```

```
float valor1, valor2, valor3, total;
valor1 = 12.6;
valor2 = 3.4;
valor3 = 6.2;
total = valor1 + valor2 + valor3;
printf("Total: %.2f \n", total);
```

```
// terceira atividade
```

```
int idade;
float altura;
char inicial;
```

```
printf("Digite sua idade: ");
scanf("%d", &idade);
```

```
printf("Digite sua altura: ");
scanf("%f", &altura);
```

```
printf("Digite a inicial do seu nome: ");
scanf(" %c", &inicial);
```

```
printf("\nIdade: %d, Altura: %.2f, Inicial: %c\n ", idade, altura, inicial);
```

```
// variáveis
char name[50];
char email[50];
int age;
```

```
// leitura de dados
printf("Digite seu nome: ");
scanf("%s", &name);
```

```
printf("Digite sua idade: ");
scanf("%d", &age);
```

```
printf("Digite seu e-mail: ");
scanf("%s", &email);
```

```
// mostrando os dados
printf("Ola, %s! Voce tem %d anos, seu e-mail: %s\n", name, age, email);
```

```
return 0;
```

```
}
```

Referências

<https://lugarcerto.wordpress.com/2013/03/29/estrutura-basica-de-um-programa-em-c/>

<https://linguagemc.com.br/tipos-de-dados-em-c/>

<https://linguagemc.com.br/variaveis-em-linguagem-c/>

<https://learn.microsoft.com/pt-br/cpp/c-language/c-identifiers?view=msvc-170>

<https://www.inf.pucrs.br/~pinho/Laprol/IntroC/IntroC.htm>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Operadores Básicos em C	Relatório 2
--	---	--------------------

Operadores de atribuição

Operadores de atribuição atribuem o valor do operando à direita ao operando da esquerda, que é quem recebe o valor.

Exemplos: = *= /= %= += -= <=> &= ^= !=

Esses operadores de atribuição em C, podem atribuir valores ou transformá-los.

Operadores aritméticos em C

- + (soma), exemplo: A + B
- - (subtração), exemplo: A - B
- * (multiplicação), exemplo: A * B
- / (divisão inteira), exemplo: B / A
- % (módulo), exemplo: B % A

Operadores relacionais e lógicos

Relacionais:

- == (igual), exemplo: A == B
- != (diferente), exemplo: A != B
- > (maior do que), exemplo: A > B
- < (menor do que), exemplo: A < B
- >= maior ou igual, exemplo: A >= B
- <= menor ou igual, exemplo: A <= B

Lógicos:

- && (AND lógico), exemplo: A && B
- || (OR lógico), exemplo: A || B
- ! (NOT lógico), exemplo: !A

Operadores incremento e decremento

- ++ (incremento), exemplo: A++
- -- (decremento), exemplo: A--

Os operadores de incremento e decremento tem os pré e os pós-fixados, no pré (++a)

o incremento acontece antes que o valor seja usado, no pós (a++) acontece depois que o valor for utilizado.

Códigos-Fontes do roteiro, exemplos e atividades

```
// aula 2
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    setlocale(LC_ALL, "Portuguese");

    // primeira atividade

    int a = 10, b = 3;

    printf("Soma: %d\n", a + b);
    printf("Subtracao: %d\n", a - b);
    printf("Multiplicacao: %d\n", a * b);
    printf("Divisao: %d\n", a / b);
    printf("Resto da divisao: %d\n", a % b);

    // segunda atividade

    int x = 5;

    x += 3;
    printf("Resultado de x += 3: %d\n", x);

    x *= 2;
    printf("Resultado de x *= 2: %d\n", x);

    // terceira atividade

    int idade = 18;

    printf("Idade igual a 18? %d\n", idade == 18);
    printf("Idade maior que 17 %d\n", idade > 17);
    printf("Idade diferente de 20? %d\n", idade != 20);

    // quarta atividade
    int A = 5, B = 10;

    printf("a > 0 E b > 0: %d\n", (A > 0) && (B > 0));
    printf("a > 0 OU b < 0: %d\n", (A > 0) || (B < 0));
```

```
printf("Nao (a > b): %d\n", !(A > B));

// quinta atividade

// variáveis
char name[50];
int age;
float n1, n2, media;
// entrada de dados
printf("Digite seu nome: ");
scanf(" %[^\\n]", &name);

printf("Digite sua idade: ");
scanf("%d", &age);

printf("Digite duas notas: ");
scanf("%f %f", &n1, &n2);

media = (n1 + n2 ) / 2;
// resultado
printf("\\Ola, %s!\\n", name);
printf("Media: %.2f\\n", media);
printf("Media >= 7? %d\\n", media >= 7);
printf("Idade < 18? %d\\n", age < 18);

return 0;

}
```

Referências

<https://learn.microsoft.com/pt-br/cpp/c-language/c-assignment-operators?view=msvc-170>

https://www.inf.ufpr.br/hexsel/ci067/02_operad.html

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Operações e Controle de Fluxo	Relatório 3
--	---	--------------------

Estruturas condicionais if

Estruturas condicionais são o que dão a capacidade ao programa de alterar o modo que o código irá se comportar dependendo das condições. E é para isso que utilizamos o if, ele irá tomar uma decisão com base em uma condição ou expressão, se a condição for verdadeira irá executar aquele bloco de código que está dentro dele, se for falsa aquele bloco vai ser ignorado.

Estruturas condicionais if-else

If-else funciona de maneira similar, porém se a condição do if for falsa, ele irá executar a que está dentro do else, caso seja verdadeira executa o que está dentro do if. Dessa maneira conseguimos construir programas mais robustos e complexos, que podem tomar decisões de diferentes em cenários diferentes.

Códigos-Fontes do roteiro, exemplos e atividades

Atividade proposta 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    // Variáveis

    char name[50];
    int age;
    float n1;
    float n2;
    float n3;
    float average;
```

```
// Pegando dados
```

```
printf("Coloque seu nome: ");  
scanf("%s", &name);
```

```
printf("Coloque sua idade: ");  
scanf("%d", &age);
```

```
printf("Coloque sua primeira nota: ");  
scanf("%f", &n1);
```

```
printf("Coloque sua segunda nota: ");  
scanf("%f", &n2);
```

```
printf("Coloque sua terceira nota: ");  
scanf("%f", &n3);
```

```
average = (n1 + n2 + n3) / 3;
```

```
// dependendo da média a mensagem executada será diferente
```

```
if (average >= 9){  
    printf("Aprovado com excelente desempenho!");  
} else if (average >= 7){  
    printf("Aprovado, mas ainda pode melhorar.");  
} else {  
    printf("Reprovado. Continue se esforçando!");  
}
```

```
return 0;
```

```
}
```

Exemplo 1:

```
#include <stdio.h>  
#include <stdlib.h>  
#include <locale.h>
```

```
int main()  
{
```

```

// Atividade 1
setlocale(LC_ALL, "Portuguese");

int idade;
printf("Digite sua idade: ");
scanf("%d", &idade);
// Mensagem caso idade seja maior ou igual a 18
if(idade >= 18){
    printf("Voce é maior de idade.\n");
}

return 0;
}

```

Exemplo 2:

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    setlocale(LC_ALL, "Portuguese");

    // Atividade 2

    int numero;
    printf("Digite um número: ");
    scanf("%d", &numero);
    // Verificação se o número é par ou impar
    if(numero % 2 == 0){
        printf("Par");
    } else {
        printf("Impar");
    }

    return 0;
}

```

Exemplo 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    // Atividade 3

    int senhaDigitada;
    int senhaCorreta = 1234;

    printf("\nDigite a senha: ");
    scanf("%d", &senhaDigitada);
    // Caso senha diferente, acesso negado
    if(senhaDigitada == senhaCorreta){
        printf("Senha permitida.\n");
    } else {
        printf("Senha incorreta! tente novamente.\n");
    }

    return 0;
}
```

Exemplo 4:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    setlocale(LC_ALL, "Portuguese");
}
```

```
// Variável do tipo int
int nota;

printf("Digite a nota (0 a 10): ");
scanf("%d", &nota);
// nota maior ou igual a 7 está aprovado, entre 5 e 6 recuperação, menor que 5
reprovado
if (nota >= 7){
    printf("Aprovado.\n");
} else if (nota >= 5){
    printf("Em recuperação.\n");
} else {
    printf("Reprovado.\n");
}

return 0;
}
```

Referências

<https://www.freecodecamp.org/portuguese/news/instrucoes-if-else-em-c-explicadas/>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Estruturas de Repetição em C (Laços)	Relatório 4
--	--	--------------------

Laços de repetição: while, do-while e for

While

Irá executar um bloco de código enquanto a condição for verdadeira, se for falsa desde o início, o laço será ignorado. A variável que vai ser testada dentro da condição, chamamos de variável de controle, quando ela se tornar falsa irá pular para fora do laço.

do-while

Com o do-while diferente do while, o bloco de código será executado pelo menos uma vez, mesmo que a condição seja falsa desde o início, isso acontece, pois a condição é testada apenas no final.

for

Muito utilizada quando se sabe quantas vezes irá se repetir o laço, esse laço utiliza uma variável para controlar a contagem do loop e também seu incremento. Essa estrutura faz a inicialização, incremento e encerramento do laço.

Break e Continue

O comando break ele cumpri a função de encerrar um laço completamente, o comando Continue ele não encerra o loop e sim pula para a próxima iteração, ignorando os comandos intermediários.

Switch-case

É uma maneira de reduzir a complexidade de vários if e else encadeados, caso a condição que está sendo testada for verdadeira o bloco de código é executado. O comando break termina a execução do switch.

Códigos-Fontes do roteiro, exemplos e atividades

Atividade 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
```

```
setlocale(LC_ALL, "Portuguese");

// Variáveis que vão ser utilizadas durante a execução do programa
float numero;
float soma;
float media;
int quantidade = 0;

// Vai continuar executando até que o usuário digite um número diferente de zero
while(numero != 0){
    printf("Digite um número: ");
    scanf("%f", &numero);
    if(numero > 0){
        soma += numero;
        quantidade++;
    }
}

media = soma / quantidade;
// Média formatada em duas casas decimais
printf("Media: %.2f\n", media);

return 0;
}
```

Atividade 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    // Quantidade de linhas que "árvore" vai ter
    int linhas;

    printf("Quantidade de linhas: ");
```

```
scanf("%d", &linhas);
// Árvore feita com um laço aninhado
int l, k;
for (l = 1; l <= linhas; l++){

    for (k = 1; k <= l; k++){
        printf("*");
    }
    printf("\n");
}

return 0;
}
```

Exemplo 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // Inicialização da variável
    int i = 0;
    // Loop while
    while(i <= 5){
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```

Exemplo 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    // Variável inicial
    int valor = 0;
    while(valor != -1){
        printf("Digite um número (-1 para sair): ");
        scanf("%d", &valor);
    }
    printf("Programa encerrado.\n");

    return 0;
}
```

Exemplo 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    // Inicialização da variável
    char opcao;
    // do while, vai executar pelo menos uma vez
    do {
        printf("Deseja continuar? (s/n): ");
        scanf(" %c", &opcao);
    } while (opcao == 's' || opcao == 'S');

    printf("Fim do programa.\n");

    return 0;
}
```

```
}
```

Exemplo 4:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    // Variável inicial
    int l;
    // Loop de 1 até 5
    for (l = 1; l <= 5; l++){
        printf("Valor: %d\n", l);
    }

    return 0;
}
```

Exemplo 5:

```
int o, j;

for(o = 1; o <= 10; o++){
    printf("\nTabuada do %d:\n", o);

    for(j = 1; j <= 10; j++){
        printf("%d x %d = %d\n", o, j, o * j);
    }
}
```

Referências

<https://linguagemc.com.br/o-comando-while-em-c/>

<https://linguagemc.com.br/comando-do-while/>

<https://linguagemc.com.br/a-estrutura-de-repeticao-for-em-c/>

<https://linguagemc.com.br/o-comando-continue/>

<https://linguagemc.com.br/o-comando-switch-case-em-c/>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Vetores (Arrays Unidimensionais)	Relatório 5
--	--	--------------------

Vetores (arrays unidimensionais)

Vetores é uma estrutura de dados que armazena uma quantidade de dados do mesmo tipo, para localizar a posição que um item está dentro de um vetor, usamos o índice, que é um número inteiro que guarda a posição dos itens. Utilizando eles conseguimos manipular uma grande quantidade de dados com apenas uma variável.

Operações com vetores

Exemplo de declaração de um vetor com 10 itens do tipo float: float vet[10], podemos também declarar e inicializar um vetor dessa forma: int numeros[5] = {1, 2, 3, 4, 5}. Nos códigos fonte abaixo, você verá diversos tipos de exemplos utilizando vetores.

Códigos-Fontes do roteiro, exemplos e atividades

Demonstração prática e exemplo 1 :

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // Demonstração prática
    int not[5];
    not[0] = 1;
    not[1] = 2;
    not[2] = 3;
    not[3] = 4;
    not[4] = 5;
    // exemplo 1
    int idades[3] = {10, 20, 30};
    return 0;
}
```

Exemplo 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    int notas[2];
    notas[0] = 10;
    notas[1] = 30;

    return 0;
}
```

Exemplo 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // exerc 3
    int tam;

    printf("Digite o tamanho do vetor: ");
    scanf("%d", &tam);

    int vetor_exemplo[tam];

    return 0;
}
```

Exemplo 4, 5 e 6:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    //exemplo 4
    int i, vetor[10];
```

```

for(i = 0; i < 10; i++){
    printf("Digite o %d valor: ", i+1);
    scanf("%d", &vetor[i]);
}

//exemplo 5
printf("%d\n", vetor[0]);
printf("%d\n", vetor[1]);

//Exemplo 6

int tamanho = sizeof(vetor)/ sizeof(vetor[0]);

int j = 0;
for (j; j < tamanho; j++){
    printf("%d ", vetor[j] );
}
return 0;
}

```

Exemplo 7:

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    int nota[5];

    nota[0] = 10;
    nota[1] = 8;
    nota[2] = 9;
    nota[3] = 7;
    nota[4] = 6;

    printf("Nota da terceira posição: %d\n", nota[2]);
    return 0;
}

```

Exemplo 8:

```

#include <stdio.h>

```

```

#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    int t, tamanho_vetor = 3;
    int array[tamanho_vetor];

    array[0] = 1;
    array[1] = 2;
    array[2] = 3;

    for (t = 0; t < tamanho_vetor; t++){
        printf("%d ", array[t]);
    }

    return 0;
}

```

Exemplo 9:

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    int s, valores[5], soma = 0;
    float media;

    for(s = 0; s < 5; s++){
        printf("\nDigite o valor %d: ", s + 1);
        scanf("%d", &valores[s]);
        soma += valores[s];
    }

    media = soma / 5.0;

    printf("\nSoma: %d\n", soma);
    printf("\nMédia: %.2f\n", media);
}

```

```
return 0;

}
```

Atividade Proposta 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // Variáveis
    int vet[10], total = 0, v;
    float media_vet;
    // Armazenando os dados
    for (v = 0; v < 10; v++){
        printf("\nDigite um valor: ");
        scanf("%d", &vet[v]);
        total += vet[v];
    }
    // Média
    media_vet = total / 10.0;

    printf("\nMédia: %.2f", media_vet);
    printf("\nSoma: %d", total);
    printf("\nValores pares: ");

    //Números pares armazenados no vetor
    int n;
    for (n = 0; n < 10; n++){
        if(vet[n] % 2 == 0){
            printf("%d ", vet[n]);
        }
    }

    return 0;
}
```

Referências

<https://linguagemc.com.br/vetores-ou-arrays-em-linguagem-c/>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Matrizes (Arrays Bidimensionais)	Relatório 6
--	--	--------------------

Matrizes (arrays bidimensionais)

Também chamada de Matriz 2D, é uma tabela de linhas e colunas, em C pode ser criada dessa maneira: `int matriz[3][2] = {{1, 2}, {3, 4}, {5, 6}};`

A primeira dimensão é a quantidade de linhas, no exemplo seria [3] linhas, a segunda é a quantidade de colunas que seriam [2].

Manipulação de matrizes: acesso e operações

Para acessar um valor armazenado dentro de uma matriz, deve indicar o número do índice da linha e da coluna, como nesse exemplo: `matriz[0][1]`.

Para atribuir um valor ao local dentro da matriz em C, pode ser feito dessa maneira: `matriz [0][0] = 10;`

Códigos-Fontes do roteiro, exemplos e atividades

Atividade proposta 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    // declaração de variáveis

    int matriz_usuario[3][3];
    int L, C;
    int valor;

    int soma_total = 0;
    int soma_par = 0;
    float media;

    // for para atribuir os valores a matriz
```

```

for (L = 0; L < 3; L++){
    for(C = 0; C < 3; C++ ) {

        printf("Digite o valor que vai na linha %d e coluna %d: ", L, C);
        scanf("%d", &valor);
        matriz_usuario[L][C] = valor;
    }
}

printf("Matriz criada: \n");
//Mostrando a matriz criada e somando o total dos valores armazenados dentro dela e a
soma dos valores pares
for (L = 0; L < 3; L++){
    for(C = 0; C < 3; C++ ) {

        printf("%d ", matriz_usuario[L][C]);

        soma_total += matriz_usuario[L][C];

        if (matriz_usuario[L][C] % 2 == 0) {

            soma_par += matriz_usuario[L][C];

        }
    }
    printf("\n");
}
// media de todos valores, divida pelo total de elementos, linhas x colunas

media = soma_total / 9.0;

printf("Soma total: %d\n", soma_total);
printf("Soma dos pares: %d\n", soma_par);
printf("Media: %.2f\n", media);

return 0;
}

```

Exemplo 1

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    // matriz criada
    int m[3][4] = {{3, 4, 5, 5}, {1, 3, 6, 5}, {8, 1, 2, 5}};

    int i, j;
    for (i = 0; i < 3; i++){
        for(j=0; j<4; j++){
            //imprimindo a matriz
            printf("%d ", m[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Exemplo 2

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    // VARIÁVEIS
    int lin = 5, col = 2;
    int matriz[5][2];
    int l, c;

    // Atribuindo valores
    for(l = 0; l < lin; l++){
```

```

        for(c = 0; c < col; c++){
            matriz[l][c] = l + c;
        }
    }
    // Mostrando a matriz
    printf("Matriz preenchida com l + c:\n");
    for(l = 0; l < lin; l++){
        for(c = 0; c < col; c++){
            printf("%d ", matriz[l][c]);
        }
        printf("\n");
    }

    return 0;
}

```

Exemplo 3

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    //Matriz 3 por 3
    int mat[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9},
    };

    int l, j, soma = 0;
    // Fazendo a soma dos valores
    for(l = 0; l < 3; l++){
        for(j = 0; j < 3; j++){
            soma += mat[l][j];
        }
    }

    printf("Soma dos elementos da matriz: %d \n", soma);
}

```

```
    return 0;  
}
```

Referências

https://www-w3schools-com.translate.goog/c/c_arrays_multi.php?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Funções em C	Relatório 7
--	--	--------------------

Funções em C

Em C uma função é um conjunto de comandos que realiza uma tarefa. Ela é referenciada no programa principal pelo nome que foi atribuído para ela, quando utilizada ela modulariza o programa.

Chamada de funções e passagem de parâmetros em C

Os parâmetros são variáveis que são declaradas no cabeçalho da função. Nesse exemplo os parâmetros seriam: int x e int y:

```
int calcular_valores(int x, int y){
    return x + y;
}
```

A chamada da função pode ser feito da seguinte maneira:

```
int soma = calcular_valores(2, 3);
```

O resultado que a função retornar será armazenado dentro da variável soma.

Variáveis locais

São declaradas dentro de um bloco de uma função, não podem ser acessadas em outras funções, só irá existir enquanto a função em que foi declarada está sendo executada.

Variáveis globais

São declaradas fora de todas as funções, são acessíveis em todas as funções do programa e vão existir durante a execução de todo programa.

Códigos-Fontes do roteiro, exemplos e atividades

Atividade proposta 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>

int contar_caracteres(char nome[]);
```

```

int contar_caracteres(char nome[]) {

    int i = 0;
    // conta espaço entre nome e sobrenome
    while(nome[i] != '\0'){
        i++;
    }
    return i;

}

int main()
{
    setlocale(LC_ALL, "Portuguese");

    char nome[50];
    printf("Digite seu nome: ");
    scanf(" %[^\\n]", nome); // le nome com espaços

    int total = contar_caracteres(nome);
    printf("O nome possui %d caracteres.\\n", total);

    return 0;

}

```

Exemplo 1:

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>

int calcula_quadrado(int num);
// Função calcula_quadrado recebe um número inteiro e retorna o resultado do número
vezes ele mesmo
int calcula_quadrado(int num) {
    return num * num;
}

```

```

int main()
{
    setlocale(LC_ALL, "Portuguese");

    //Variáveis
    int num = 10;

    int n1 = 4;

    int quad = calcula_quadrado(n1);

    printf("%d\n", quad);

    return 0;
}

```

Exemplo 2:

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>

void modifReferencia(int *x);
// função que muda o valor da variável pelo ponteiro
void modifReferencia(int *x){
    printf("Dentro da função, antes da modificação: *ptr = %d\n", *x);
    *x = 100;
    printf("Dentro da função, depois da modificação: *ptr = %d\n", *x);
}

int main()
{
    setlocale(LC_ALL, "Portuguese");
    //Variável
    int valor = 10;

    printf("Antes da chamada da função: valor = %d\n", valor);
}

```

```
modifReferencia(&valor);

printf("Depois da chamada da função: valor = %d\n", valor);

return 0;

}
```

Exemplo 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>

// Variável global
double resultado;
// Protótipos
void calculaquadrado(double num);
double calculasoma(double n1, double n2);
// Funções
void calculaquadrado(double num){
    resultado = num * num;
}

double calculasoma(double n1, double n2){
    double r;
    r = n1 + n2;
    return r;
}

int main()
{
    setlocale(LC_ALL, "Portuguese");
    //Variável
    int valor = 10;

    printf("Antes da chamada da função: valor = %d\n", valor);

    modifReferencia(&valor);

    printf("Depois da chamada da função: valor = %d\n", valor);
}
```

```
return 0;  
}
```

Referências

<https://linguagemc.com.br/funcoes-em-c/>

<https://linguagemc.com.br/funcoes-e-escopo-de-variaveis/>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Ponteiros e Manipulação de Arquivos	Relatório 8
--	---	--------------------

Definição de ponteiros em C

Ponteiros são variáveis que armazenam o endereço de memória de outra variável, são úteis quando uma variável tem que ser acessada em diferentes partes do programa. Exemplo de uso na linguagem C:

```
int valor = 20;
int *ponteiro;
ponteiro = &valor;
```

Manipulação de arquivos (leitura e escrita em arquivos)

Com a manipulação de arquivos podemos armazenar arquivos de maneira permanente, assim mesmo que o programa já tenha sido encerrado elas ainda ficam disponíveis.

Algumas funções básicas em C para manipulação de arquivos:

- fopen() - para criar um arquivo ou abrir um
- fclose() - para fechar um arquivo
- fscanf() - para ler um conjunto de dados
- fprintf() - para escrever um conjunto de dados

Códigos-Fontes do roteiro, exemplos e atividades

Atividade proposta 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // Variáveis
    char nome[50];
    int idade;
```

```

// Armazenando dados nas variáveis
printf("Seu nome: ");
fgets(nome, sizeof(nome), stdin);

printf("Sua idade: ");
scanf("%d", &idade);
//Abrindo arquivo
FILE *arquivo_usuario = fopen("cadastro.txt", "w");
//Escrevendo dentro dele
if (arquivo_usuario != NULL) {
    fprintf(arquivo_usuario, "Nome: %sIdade: %d", nome, idade);
    fclose(arquivo_usuario);
    printf("Arquivo escrito com sucesso.\n");
} else {
    printf("Erro ao abrir o arquivo.\n");
}

// Leitura de arquivos
FILE *leitura = fopen("cadastro.txt", "r");
char linhas[100];
printf("Leitura do arquivo\n");
if (arquivo_usuario != NULL) {
    while(fgets(linhas, 100, arquivo_usuario) != NULL){
        printf("%s", linhas);
    }
} else {
    printf("Error ao abrir o arquivo.\n");
}

return 0;
}

```

Exemplo 1:

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
int main()
{

```

```
setlocale(LC_ALL, "Portuguese");
// Variáveis
int x = 10;

int *p;

p = &x; // vai receber o endereço de x

printf("Valor de x = %d\n", x);
printf("Endereço de x = %p\n", &x);
printf("Endereço armazenado em p = %p\n", p);
printf("Valor apontado por p = %d\n", *p);

return 0;
}
```

Exemplo 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // Variáveis
    int X = 10;
    int *P = &X;
    // Valores dentro dela
    printf("Valor de x: %d\n", X);
    printf("Endereço de x: %p\n", &X);
    printf("Valor apontado por p: %d\n", *P);

    *P = 200;

    printf("Novo valor de x: %d\n", X);
    printf("Valor apontado por p: %d\n", *P);

    return 0;
}
```

Exemplo 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // Abrindo arquivo
    FILE *arquivo = fopen("dados.txt", "w");
    // Escrevendo dentro dele
    if(arquivo != NULL){
        fprintf(arquivo, "Olá, mundo!\n");
        fclose(arquivo);
        printf("Arquivo escrito com sucesso.\n");
    }
    else {
        printf("Erro ao abrir o arquivo.\n");
    }

    return 0;
}
```

Exemplo 4:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // Abrindo arquivo
    FILE *arq = fopen("dados.txt", "r");
    char linha[100];
    // Lendo o que está escrito dentro dele
    if(arq != NULL) {
        while(fgets(linha, 100, arq) != NULL){
            printf("%s", linha);
        }
    }
}
```

```
    fclose(arq);  
} else {  
    printf("Erro ao ler o arquivo.\n");  
}  
  
return 0;  
}
```

Referências

<https://linguagemc.com.br/ponteiros-em-c/>

<https://linguagemc.com.br/arquivos-em-c-categoria-usando-arquivos/>

<https://www.freecodecamp.org/portuguese/news/manipulacao-de-arquivos-em-c-como-abrir-e-fechar-arquivos-e-escrever-algo-neles/>